

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 04 -

ÁRVORES DE DECISÃO E ENSEMBLES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	10
MERCADO, CASES E TENDÊNCIAS	21
O QUE VOCÊ VIU NESTA AULA?	26
REFERÊNCIAS.....	28

EMANIP

O QUE VEM POR AÍ?

Imagine que você trabalha em uma operadora de telecomunicações e precisa desenvolver um modelo para prever quais clientes estão prestes a cancelar o serviço (churn). Você começa pelo básico, utilizando uma árvore de decisão por sua interpretabilidade e clareza nas regras geradas, identificando padrões como atrasos frequentes de pagamento e pouco tempo restante de contrato. Com o uso, porém, surge um dilema clássico: árvores muito complexas tendem ao overfitting, perdendo desempenho em novos dados, enquanto árvores excessivamente restritas sofrem de underfitting, deixando de capturar relações relevantes. Esse equilíbrio delicado entre simplicidade e capacidade de generalização motiva a busca por abordagens mais robustas.

É nesse contexto que entram os métodos Ensemble, que combinam múltiplos modelos para produzir previsões mais estáveis e precisas, de forma análoga a consultar vários especialistas em vez de confiar em apenas um. Nossa aula explora dois paradigmas centrais dessa abordagem: o Bagging, com destaque para as Florestas Aleatórias, e o Boosting, incluindo o Gradient Boosting, base de algoritmos amplamente utilizados como XGBoost e LightGBM. Antes disso, são revisitados os fundamentos das árvores de decisão, seus critérios de impureza e as razões pelas quais apresentam alta variância, além de estratégias como poda e parâmetros de parada para controlar a complexidade.

Na sequência, o Bagging é apresentado como uma técnica que reduz a variância ao treinar múltiplas árvores independentes em diferentes amostras dos dados e agregar suas previsões, sendo a Random Forest um caso especial que adiciona aleatoriedade na seleção de atributos. Em contraste, o Boosting constrói modelos de forma sequencial, concentrando-se progressivamente nos erros anteriores para reduzir o viés e alcançar alta acurácia. Esses conceitos são acompanhados de exemplos práticos em Python, discussão de hiperparâmetros e boas práticas de validação, culminando em uma visão aplicada do uso de ensembles em áreas como telecomunicações, finanças e nas principais tendências do mercado em Machine Learning. Preparado(a)? Então vamos mergulhar no universo das árvores de decisão e suas poderosas florestas de modelos combinados!

HANDS ON

Agora que você já tem uma ideia dos conceitos iniciais, vamos partir para a prática com pequenos experimentos em Python. Nestas atividades hands on, construiremos modelos de árvore de decisão, Random Forest e Gradient Boosting, usando dados de exemplo para comparar seus desempenhos e entender suas características. Os códigos foram testados para garantir que são funcionais, servindo para ilustrar as ideias discutidas.

Árvore de decisão simples – conceitos básicos e overfitting

Começaremos treinando uma árvore de decisão de classificação simples utilizando a biblioteca scikit-learn. Vamos usar um conjunto de dados clássico, o Iris, que possui 150 amostras de flores de três espécies, descritas por quatro medidas de características. Nosso objetivo será prever a espécie da flor com base nas medidas das pétalas e sépalas. Primeiro, iremos treinar uma árvore de decisão sem restrições específicas e avaliar sua performance:

```
1  from sklearn.datasets import load_iris
2  from sklearn.model_selection import train_test_split
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.metrics import accuracy_score,
confusion_matrix
5
6  # Carrega dados de exemplo (flores Iris)
7  iris = load_iris()
8  X_train, X_test, y_train, y_test = train_test_split(
9      iris.data, iris.target, test_size=0.3,
stratify=iris.target, random_state=42
10 )
11
12 # Cria e treina uma Árvore de Decisão sem
restrições de profundidade
13 modelo_arvore =
DecisionTreeClassifier(random_state=42)
14 modelo_arvore.fit(X_train, y_train)
15
16 # Realiza previsões no conjunto de teste
17 y_pred = modelo_arvore.predict(X_test)
18
19 # Avalia a acurácia e gera a matriz de confusão
20 acuracia = accuracy_score(y_test, y_pred)
```

```
21     mat_confusao = confusion_matrix(y_test, y_pred)
22     print(f"Acurácia da Árvore de Decisão:
{acuracia:.2f}")
23     print("Matriz de Confusão:\n", mat_confusao)
```

Código-fonte 1 - from sklearn.datasets import load_iris
Fonte: Elaborado pelo autor (2026)

Ao executar esse código, a saída indica algo como:

```
1     Acurácia da Árvore de Decisão: 0.98
2     Matriz de Confusão:
3     [[19  0  0]
4      [ 0 13  2]
5      [ 0  0 11]]
```

Código-fonte 2 – Saída com matriz de confusão (Python)
Fonte: Elaborado pelo autor (2026)

No nosso exemplo, a árvore de decisão acertou cerca de 98% das amostras de teste, o que é excelente. No entanto, devemos tomar cuidado com resultados aparentemente perfeitos: muitas vezes, uma árvore muito profunda pode estar se superajustando aos dados de treino, aprendendo ruídos em vez de padrões gerais. Uma inspeção mais profunda da árvore (usando, por exemplo, `modelo_arvore.get_depth()`) revelaria se ela atingiu grande profundidade. Para mitigar o overfitting, poderíamos limitar a profundidade máxima da árvore (`max_depth`), exigir um número mínimo de amostras para dividir um nó (`min_samples_split`), ou para definir uma folha (`min_samples_leaf`), ou aplicar poda posterior se a árvore já estiver totalmente crescida. Essas técnicas de pruning efetivamente removem ramificações irrelevantes ou pouco generalizáveis, diminuindo a complexidade do modelo e melhorando sua capacidade de generalização.

Além disso, explorando os critérios de divisão da árvore, podemos escolher entre Índice Gini (valor padrão no scikit-learn) ou Ganho de Informação (Entropia). Ambos medem o quão “puro” (homogêneo) fica cada nó após uma possível divisão. No critério Gini, buscamos minimizar a probabilidade de uma classificação incorreta ao escolher um ramo aleatoriamente no nó. Já a Entropia, inspirada na teoria da informação, mede o grau de desordem do nó e atinge 0 quando o nó é puro (todas as instâncias da mesma classe) e valor máximo quando as classes estão uniformemente misturadas.

O algoritmo de indução de árvores calcula o ganho de informação resultante de usar cada atributo como divisor e escolhe aquele que maximiza a redução de impureza (maior ganho de informação). Assim, a árvore vai sendo construída adicionando, em cada passo, o atributo que melhor separa as classes em termos de pureza. Esses critérios levam a escolhas ligeiramente diferentes: na prática, o Índice Gini costuma isolar a classe mais frequente em um dos ramos, enquanto a Entropia tende a criar divisões que balanceiam melhor as proporções em ambos os lados.

Random forest – reduzindo a variância com bagging

Agora, vamos ver na prática os benefícios do Bagging treinando um modelo de Random Forest sobre o mesmo conjunto de dados. O Random Forest, como vimos, constrói várias árvores de decisão em paralelo, cada uma em uma amostra de dados obtida por bootstrap (amostragem aleatória com reposição), e combina seus resultados por votação majoritária (no caso de classificação) ou média (no caso de regressão). Além disso, em cada nó de divisão, a Random Forest seleciona aleatoriamente apenas um subconjunto de atributos para avaliar a melhor divisão, injetando ruído estrutural no processo e diminuindo a correlação entre as árvores. Vamos treinar uma Random Forest com 100 árvores (`n_estimators=100`) e comparar sua performance com a árvore simples:

```
1     from sklearn.ensemble import RandomForestClassifier
2
3     # Instancia e treina uma Random Forest com 100
árvores (estimadores)
4     modelo_rf = RandomForestClassifier(n_estimators=100,
random_state=42)
5     modelo_rf.fit(X_train, y_train)
6
7     # Avalia o desempenho no conjunto de teste
8     y_pred_rf = modelo_rf.predict(X_test)
9     acuracia_rf = accuracy_score(y_test, y_pred_rf)
10    print(f"Acurácia da Random Forest:
{acuracia_rf:.2f}")
```

Código-fonte 3 - from sklearn.ensemble import RandomForestClassifier

Fonte: Elaborado pelo autor (2026)

Ao rodar o código, podemos obter, por exemplo:

Acurácia da Random Forest: 1.00

No caso do conjunto de dados Iris, a Random Forest também consegue classificar corretamente todas as amostras de teste (100% de acerto). Isso não chega a surpreender, pois o problema é relativamente simples e nossas árvores de decisão individuais já tinham desempenho próximo do ideal. A vantagem da Random Forest aparece especialmente em problemas mais complexos e ruidosos, em que cada árvore isolada teria um desempenho bem variável.

Nesses cenários, a agregação das árvores via bagging costuma reduzir a variância dos erros: erros cometidos por algumas árvores são compensados por acertos de outras, produzindo um resultado mais estável. Em testes com dados mais complexos, é comum uma Random Forest superar uma única árvore; mesmo que cada árvore possa superestimar padrões específicos (sobreajustando), na média esse efeito é atenuado. Além disso, a porção de dados não usada em cada bootstrap serve como um conjunto de validação interno (amostra out-of-bag), permitindo estimar a acurácia de generalização da floresta sem necessidade de um conjunto de validação separado.

Outra vantagem das Random Forests é que elas facilitam a análise de importância de atributos (feature importance). Como veremos adiante, muitas implementações (como a do scikit-learn) fornecem automaticamente uma medida de quão relevante cada variável foi para as decisões do modelo. Isso ajuda a interpretar, em alguma medida, o que o ensemble “aprendeu” – por exemplo, identificando quais fatores mais contribuem para a decisão de churn de um cliente. Contudo, ainda assim, uma floresta com centenas de árvores é bem menos interpretável do que uma única árvore de decisão.

Há um trade-off entre precisão e interpretabilidade: modelos de árvore única são transparentes, porém potencialmente menos precisos, enquanto ensembles de árvores podem atingir alta performance ao custo de serem tratados como caixas-pretas. No contexto de problemas que exigem explicações claras (como saúde ou finanças), essa falta de transparência pode ser um impeditivo e por isso métodos como regressão logística ou árvores individuais ainda são preferidos nesses casos – ou então emprega-se técnicas de interpretação de modelos complexos (ex.: cálculo de Shapley values em ensembles de árvores).

Gradient Boosting – Combinação Sequencial e Redução de Viés

Por fim, vamos experimentar a família de métodos de Boosting, que adotam uma estratégia bem diferente: em vez de treinar modelos independentes em paralelo, se treinam modelos de forma sequencial, cada um corrigindo os erros do anterior. Aqui usaremos a técnica de Gradient Boosting, que generaliza o AdaBoost para otimizar erros residuais através de gradientes (teremos mais detalhes na próxima seção). Vamos treinar um modelo de Gradient Boosting Classifier no mesmo conjunto de dados e comparar a performance:

```
1 from sklearn.ensemble import
  GradientBoostingClassifier
2
3 # Treina um modelo de Gradient Boosting (100 árvores
  de profundidade 3 padrão)
4 modelo_gb =
  GradientBoostingClassifier(n_estimators=100,
    learning_rate=0.1, random_state=42)
5 modelo_gb.fit(X_train, y_train)
6
7 # Avalia a performance no conjunto de teste
8 y_pred_gb = modelo_gb.predict(X_test)
9 acuracia_gb = accuracy_score(y_test, y_pred_gb)
10 print(f"Acurácia do Gradient Boosting:
    {acuracia_gb:.2f}")
```

Código-fonte 4 - from sklearn.ensemble import GradientBoostingClassifier

Fonte: Elaborado pelo autor (2026)

No nosso exemplo do Iris, o Gradient Boosting pode obter um desempenho semelhante ao da Random Forest (por exemplo, 100% de acerto no teste), já que ambos conseguem modelar bem esse conjunto simplificado. Entretanto, em problemas desafiadores, é comum que técnicas de boosting alcancem a maior acurácia dentre os métodos baseados em árvores. Isso ocorre porque o boosting reduz o viés do modelo: as primeiras árvores do conjunto capturam padrões globais e, progressivamente, as árvores subsequentes refinam casos mais difíceis, focando nas instâncias antes preditas incorretamente.

Diferentemente do bagging, que usa tipicamente modelos complexos (árvores profundas) para diminuir a variância, o boosting costuma utilizar modelos-base mais simples (por exemplo, árvores de decisão rasas, com poucas divisões) e então

aumentar sua complexidade ao combiná-los de forma aditiva. Em essência, o primeiro modelo “prepara o caminho” e os seguintes vão complementando-o, cada um corrigindo erros do anterior, resultando em um modelo final altamente acurado.

No entanto, o boosting também tem seus desafios. Por concentrar tanto poder de modelagem (várias árvores) em corrigir cada vez melhor os dados de treinamento, ele pode se tornar suscetível a overfitting se não controlado. Por isso, são introduzidos hiperparâmetros de regularização para o Gradient Boosting que nos ajudam a moderar seu aprendizado.

Alguns dos principais incluem: o número de estimadores (`n_estimators`), que define quantas árvores serão construídas (um valor muito alto pode levar a um modelo excessivamente ajustado); a taxa de aprendizado (`learning_rate`), que reduz a contribuição de cada árvore adicional (valores menores de `learning_rate` tornam o aprendizado mais lento, mas frequentemente mais robusto a overfitting); a profundidade máxima das árvores (`max_depth`), geralmente configurada para um valor baixo (por exemplo, 3 a 5) para garantir que cada árvore seja um “aprendiz fraco” e evitar nós muito específicos; e as frações de subsampling (via `subsample`) — isto é, treinar cada árvore em uma amostra aleatória dos dados — que introduzem estocasticidade e ajudam a reduzir a variância do ensemble.

Além disso, limites mínimos como `min_samples_leaf` e `min_samples_split` controlam quão específicos podem ser os nós e atuam como regularizadores estruturais. Em implementações modernas de boosting (p.ex., XGBoost, LightGBM, CatBoost), há ainda outras formas de regularização, como amostragem de colunas por árvore/nível (`colsample_bytree`, `feature_fraction`), penalizações L1/L2 nos pesos do modelo (`reg_alpha`, `reg_lambda`) e estratégias de crescimento de árvore (folha-a-folha vs. nível-a-nível), que impactam a tendência ao overfitting.

SAIBA MAIS

Agora, vamos aprofundar o conhecimento sobre árvores de decisão e métodos ensemble, analisando seus fundamentos teóricos e matemáticos. Prepare-se, pois a discussão ficará mais densa tecnicamente. Veremos como as árvores tomam decisões (critérios de impureza como Gini e Entropia), como controlamos sua complexidade (profundidade máxima, critérios de parada, poda) e então entenderemos o dilema viés–variância no aprendizado de máquina e como os ensembles o abordam.

Discutiremos formalmente Bagging (bootstrap aggregating) e Boosting – suas motivações, diferenças e como ambos melhoram a generalização dos modelos. Também trataremos de hiperparâmetros e tuning, ou seja, como ajustar finamente modelos de árvore e ensemble para obter desempenho ótimo sem sobreajuste, incluindo técnicas de validação cruzada e busca em grade (grid search).

Critérios de Divisão em Árvores: Impureza de Gini e Entropia

Uma árvore de decisão constrói regras de decisão selecionando, em cada nó, o atributo que melhor separa os dados em relação às classes alvo. Essa “qualidade” da separação é medida por uma função de impureza. Duas impurezas muito utilizadas são o Índice de Gini e a Entropia (ligada ao ganho de informação). Intuitivamente, impureza quantifica o quanto há mistura de classes em um conjunto de exemplos: quanto mais misturado (50/50%), mais impuro; quanto mais homogêneo (90/10% ou 100/0%), mais puro.

- **Índice de Gini (Impureza de Gini):** representa a probabilidade de, ao escolhermos dois exemplos aleatórios do nó, eles pertencerem a classes diferentes. Matematicamente, para um nó onde a fração de exemplos da classe i é p_i , a impureza de Gini é:

$$[\text{Gini} = \sum_{i=1}^C p_i (1 - p_i) = 1 - \sum_{i=1}^C p_i^2.]$$

- No caso binário simplifica para $\text{Gini} = 2p(1-p)$. O Gini vale 0 quando o nó é puro (uma única classe, $p=0$ ou 1) e atinge seu máximo quando há mistura máxima ($p=0,5$ gera $\text{Gini}=0,5$ para duas classes).

- **Entropia:** derivada da Teoria da Informação de Shannon, mede a incerteza ou desordem do nó. Para distribuição de classes $\{p_1, \dots, p_C\}$, a entropia é dada por:

$$[\text{Entropia} = -\sum_{i=1}^C p_i \log_2 p_i.]$$

- Em problemas binários, $\text{Entropia} = -[p \log_2 p + (1-p) \log_2 (1-p)]$. Assim como o Gini, a entropia é 0 se não há incerteza nenhuma (nó puro) e atinge máximo em 1 bit quando $p=0,5$ (incerteza máxima, classes igualmente prováveis).

Ganho de informação: ao testar um atributo para divisão, a árvore calcula a impureza antes e após a divisão e avalia o ganho obtido – ou seja, a redução na impureza média ponderada dos filhos em relação ao pai. A divisão escolhida é aquela que maximiza a redução de impureza (ou equivalentemente, maximiza o ganho de informação).

Em suma, os algoritmos de indução de árvore (como CART, ID3/C4.5) buscam a melhor pergunta a fazer em cada nó – a pergunta que deixa os subconjuntos resultantes mais puros em termos de classe. Por exemplo, se estamos classificando clientes propensos a cancelar um serviço, um nó poderia perguntar “o cliente atrasou o pagamento este mês?”; se a resposta “sim” agrega um grupo muito mais homogêneo (digamos, 90% cancelam) e “não” outro grupo (talvez só 10% cancelam), essa pergunta tem alto ganho de informação.

Complexidade da Árvore: Profundidade, Paradas e Poda

Uma árvore de decisão simples cresce recursivamente até particionar completamente os dados – em última instância, cada nó folha pode conter apenas exemplos de uma classe (ou até mesmo um único exemplo). Se deixarmos a árvore crescer sem restrições, ela tenderá a se ajustar 100% aos dados de treino, levando a sobreajuste (overfitting). Isso acontece porque a árvore irá capturar mesmo oscilações e ruídos específicos daquele conjunto de treino, perdendo a capacidade de generalizar. Para evitar isso, controlamos a complexidade da árvore por critérios de parada antecipada ou por poda posterior (pruning).

Parada antecipada envolve parâmetros como: profundidade máxima da árvore (`max_depth`), número mínimo de amostras para dividir um nó (`min_samples_split`) ou mínimo de amostras em uma folha (`min_samples_leaf`). Por exemplo, se definirmos profundidade máxima = 3, a árvore pode ter no máximo 3 divisões do nó raiz até as folhas. Esses parâmetros atuam como uma regularização, impedindo divisões muito específicas e nós com poucos exemplos. Já a poda posterior consiste em gerar inicialmente a árvore completa e, em seguida, remover nós folha pouco relevantes, tipicamente aqueles cuja remoção resulta em pequeno aumento de erro no treinamento (ou melhor ainda, avaliando em um conjunto de validação). A técnica de cost-complexity pruning do CART, por exemplo, remove iterativamente folhas e ramos inteiros se isso melhora uma métrica que equilibra erro e número de nós.

Em termos práticos, limitar a profundidade ou podar a árvore aumenta viés e diminui variância do modelo: a árvore mais rasa pode não capturar todas as nuances (introduz viés de subajuste), mas em contrapartida tende a ser mais genérica (menos variação de acordo com os dados de treino). Já a árvore totalmente crescida tem variância altíssima (muda drasticamente se os dados de treino mudam um pouco), apesar de viés nulo nos dados de treino. Esse é o clássico dilema viés-variância em aprendizagem: modelos muito simples sofrem de alto viés (não aprendem bem os padrões) e modelos muito complexos sofrem de alta variância (sensíveis ao ruído e variabilidade do conjunto de treino). Uma árvore de decisão ajustada isoladamente precisa de cuidado para encontrar um ponto intermediário.

Viés vs. Variância e o Papel dos Ensembles

Os métodos ensemble surgem exatamente para amenizar esse dilema: eles combinam vários modelos de forma a reduzir a variância total sem aumentar muito o viés, ou vice-versa. A ideia central é que usar múltiplos modelos pode cancelar erros individuais de cada um, resultando em uma predição agregada mais estável e muitas vezes mais acurada. Em particular:

- Bagging (bootstrap aggregating) tendeu a ser criado para reduzir a variância de modelos de alto variância, como árvores profundas. Ao treinar várias árvores independentes (cada uma um pouco diferente devido aos

dados de treino amostrados aleatoriamente) e depois mediar/votar suas previsões, obtemos um resultado mais robusto porque os erros de cada árvore tendem a se compensar mutuamente. Com bagging, tipicamente usamos modelos complexos (baixíssimo viés), pois queremos que cada um aprenda bastante – a variância resultante será combatida pela média do conjunto.

- Boosting foi concebido para reduzir o viés de modelos de alto viés (modelos “fracos”). A estratégia aqui é treinar modelos de forma sequencial, cada novo modelo tentando corrigir os erros do conjunto anterior. Começa-se com um modelo bem simples (alto viés) e baixo custo computacional – por exemplo, uma árvore de decisão muito rasa (com 1 ou 2 níveis). Então, gradualmente, adicionam-se novos modelos focados nas instâncias que os anteriores erraram, “impulsionando” a capacidade do conjunto em acertar casos difíceis. Isso progressivamente aumenta a complexidade efetiva do modelo (reduz o viés), mas de maneira controlada.

Em vez de vários modelos independentes, no boosting os modelos dependem uns dos outros e acabam se complementando. O resultado final é uma combinação ponderada de todos (por exemplo, média ponderada das árvores, ou votação com pesos), onde os últimos modelos (os que corrigem erros) têm maior peso. O boosting, portanto, prioriza corrigir viés, mesmo que às custas de um pouco mais de variância no final. Frequentemente, alcança-se uma acurácia maior do que bagging, porém com maior risco de sobreajuste se exagerar nas iterações.

Resumidamente, bagging constrói modelos em paralelo, visando reduzir variância de modelos complexos, enquanto boosting constrói em sequência, visando reduzir viés de modelos simples, conforme resumido a seguir.

Bagging e Random Forest em detalhes

O Bagging propriamente dito cria m conjuntos de treinamento por bootstrap: cada conjunto é gerado selecionando aleatoriamente N instâncias com reposição de um dataset original de tamanho N . Assim alguns exemplos repetem,

outros ficam de fora (em média ~36,8% ficam fora de cada bootstrap). Em cada conjunto bootstrap, treinamos um modelo (ex.: uma árvore completa). Para predizer, passamos o novo exemplo a todos os m modelos e combinamos os resultados (votação ou média). Leo Breiman demonstrou que, à medida que $m \rightarrow \infty$, a convergência dessa média tende a reduzir a variância do estimador sem aumentar o viés, desde que os modelos individuais sejam “não correlacionados”. Na prática, escolher algoritmos altamente variáveis (como árvores) e fornecer a cada um dados ligeiramente diferentes via bootstrap tende a garantir essa diversidade (os modelos não cometem os mesmos erros).

A Floresta Aleatória (Random Forest), proposta por Breiman em 2001, é um aprimoramento do bagging de árvores de decisão que incorpora mais aleatoriedade. Além de usar bootstraps dos dados para cada árvore, ela impõe que, em cada nó de cada árvore, a escolha do melhor atributo seja feita dentre um subconjunto aleatório de atributos (de tamanho k) em vez de todos os atributos. Por exemplo, se temos 50 variáveis explicativas, uma árvore da floresta, ao avaliar seu nó raiz, pode considerar apenas 10 variáveis escolhidas ao acaso para determinar a divisão; no próximo nó ela escolhe outro subconjunto aleatório e assim por diante. Essa restrição aparentemente estranha na verdade aumenta a diversidade entre as árvores – evita que todas usem sempre o mesmo atributo mais preditivo na raiz, o que as tornaria parecidas. Com árvores mais diversificadas, o ganho do ensemble é maior, pois reduz-se a correlação entre os erros das árvores.

O resultado do Random Forest é que ele costuma ter ótima performance de generalização sem praticamente nenhum ajuste fino necessário. Como cada árvore cresce totalmente (profunda) mas, por conta do ensemble, o efeito de sobreajuste é amortecido pela votação média, não precisamos podá-las. De fato, árvores em uma floresta geralmente são deixadas crescer até o limite (folhas puras ou mínimo muito baixo), pois o excesso de variância delas será compensado na agregação. Em outras palavras, cada árvore “overfitada” isoladamente se torna um “vote” em um conjunto robusto e a média das overfits se torna um underfit controlado.

Uma analogia muito usada (inclusive pelo artigo de Rebeca Honório) é pensar que, ao invés de confiar em um único especialista, nós consultamos um comitê de especialistas: cada um pode ter sua visão tendenciosa dos dados, mas a decisão coletiva tende a cancelar tendenciosidades individuais e capturar a verdade de

forma mais consistente. Não é à toa que o Random Forest se tornou conhecido como um algoritmo robusto e seguro – muitas vezes é a primeira escolha como baseline poderoso em tarefas de classificação ou regressão, por entregar boa acurácia sem muito risco. Ele também traz de brinde medidas de importância das variáveis (feature importance), calculadas a partir da frequência e ganho de informação das divisões envolvendo cada variável, o que ajuda na interpretabilidade do modelo.

Exemplo numérico: suponha que uma única árvore obtém 85% de acurácia em um conjunto de teste, mas devido à variância do aprendizado, se treinássemos árvores diferentes (com perturbações nos dados), seus resultados oscilaram de 80% a 90%. Uma Random Forest de 100 árvores poderia alcançar, digamos, 90% de acurácia com muito menos oscilação – se repetirmos o treinamento da floresta, os resultados variam pouquíssimo em torno desse valor. Isso porque a acurácia média aumentou e a variabilidade diminuiu pela agregação, conforme predito teoricamente. Além disso, o Random Forest possui uma estimativa interna de erro de generalização chamada erro Out-of-Bag (OOB): como cada árvore treina com ~63% dos exemplos (bootstraps), pode-se usar os ~37% restantes (não vistos pela árvore) para testá-la. Agregando esses erros de OOB de todas as árvores, obtemos uma medida de performance quase equivalente a validação cruzada, sem precisar de um conjunto de validação separado.

Boosting e Gradient Boosting em detalhes:

O Boosting teve início com o algoritmo AdaBoost, que introduziu a ideia de ajustar iterativamente modelos dando pesos maiores às observações mal-classificadas. O AdaBoost treina uma sequência de, por exemplo, 50 “stumps” (árvores de decisão com apenas 1 nível, que são classificadores fracos). Após cada stump, ele aumenta o peso dos exemplos que aquele stump errou e treina o próximo stump com esses pesos modificados – efetivamente forçando o próximo stump a focar nas instâncias difíceis. Ao final, combina as 50 árvores ponderando cada uma pela sua acurácia (mais confiáveis têm peso maior). Ele foi muito bem-sucedido para melhorar acurácia de classificadores fracos e inaugurou a era do boosting.

O Gradient Boosting generalizou o boosting como um procedimento de otimização de uma função de perda arbitrária, usando a ideia de descida de gradiente funcional. Em vez de ajustar pesos de instância como AdaBoost, o gradient boosting ajusta os resíduos (erros): a cada etapa, treina-se um novo modelo para prever o erro residual do modelo conjunto atual e então soma-se (com um peso λ , chamada learning rate, geralmente pequeno) esse novo modelo ao conjunto. Isso é análogo a dar um passo de gradiente descendente na função de custo global. Assim, se a função objetivo for, e.g., o erro quadrático médio, o gradient boosting adiciona modelos para corrigir diferenças contínuas; se for deviance (log-loss), adiciona modelos para corrigir log-odds etc. Em termos simples, o gradient boosting em árvores treina sequencialmente várias árvores, onde cada nova árvore “aprende” a diferença entre as previsões atuais e os valores desejados (os resíduos); soma-se essa árvore às previsões e repete. Quando nenhuma melhoria significativa nos resíduos é possível, o processo para.

Uma implementação popular é o Extreme Gradient Boosting (XGBoost), apresentado em 2016, que otimizou o gradient boosting em termos de performance (computação paralela) e incluiu diversas regularizações adicionais. Hoje, XGBoost, LightGBM e CatBoost são variações state-of-the-art de gradient boosting de árvores que costumam obter resultados de ponta em competições de Machine Learning. Esses métodos incluem parâmetros de regularização específicos (por exemplo, penalização L_1 e L_2 nos pesos das folhas das árvores no XGBoost) para evitar sobreajuste mesmo com centenas de árvores.

Comparado ao Random Forest, o gradient boosting tende a alcançar acurácias superiores em muitos casos, porém exige mais cuidado na calibração. Normalmente precisamos ajustar a taxa de aprendizado λ (também chamada shrinkage – valores menores forçam mais árvores para alcançar o mesmo ajuste, mas previnem overfitting), o número de árvores (estimadores) na sequência e a profundidade máxima de cada árvore (geralmente árvores rasas, p.ex. 3 a 8 níveis, funcionam melhor no boosting). Também é comum usar subamostragem de instâncias e atributos no boosting (p. ex. treinar cada árvore em 50% dos dados sorteados e considerando 50% das variáveis a cada divisão) para induzir diversidade e conter a variância. Na prática, bibliotecas como XGBoost e LightGBM

oferecem dezenas de hiperparâmetros e encontrar a melhor configuração envolve paciência e método.

Hiperparâmetros e Tuning

Árvores de decisão e ensembles possuem vários hiperparâmetros que controlam sua forma de aprendizado e complexidade. Selecionar bem esses hiperparâmetros é crucial para obter um modelo generalizável (bom desempenho em novos dados) sem sobreajustar aos dados de treino. Aqui estão alguns dos principais hiperparâmetros e considerações de tuning:

- **Árvores de Decisão Simples:** `max_depth`, `min_samples_split`, `min_samples_leaf`, `criterion` (gini/entropy), `max_features` (porcentagem de atributos considerados em cada divisão – geralmente usado em ensembles). O tuning de uma árvore normalmente busca o ponto em que aumentar a profundidade melhora muito pouco a acurácia de validação e passa a piorar (overfitting).
- **Random Forest:** herda os parâmetros das árvores individuais (embora tipicamente árvores de RF já podem ser adotadas bem profundas sem poda) e adiciona `n_estimators` (quantidade de árvores) e `max_features` (número de atributos considerados por divisão). O parâmetro `max_features` é importante: comum usar $\sqrt{d}$ em classificação (onde d é o número de atributos) ou $d/3$ em regressão, valores que Breiman sugeriu como bons padrões. Um `n_estimators` alto tende a melhorar até certo ponto e depois estabilizar – Random Forest geralmente não overfitta ao aumentar número de árvores, mas tem retorno decrescente em ganhos após algumas centenas de árvores. Também há a opção `bootstrap=True/False` (ativar/desativar amostragem com reposição). Se `bootstrap=False`, a técnica passa a ser chamada *pasting* (usa bagging sem repetição dos exemplos). Podemos ainda usar `oob_score=True` para que o modelo calcule o score out-of-bag, facilitando avaliar a performance sem um conjunto de validação explícito.
- **Gradient Boosting (incluindo XGBoost, LightGBM):** os principais são `n_estimators`, `learning_rate`, `max_depth` (ou `num_leaves` no LightGBM),

além de subsample (fração de exemplos por árvore) e colsample_bytree (fração de atributos por árvore) – que induzem aleatoriedade estilo RF no boosting. Aqui o trade-off central é entre `n_estimators` e `learning_rate`: um λ (`learning_rate`) menor requer mais árvores para chegar a um certo nível de ajuste, mas tende a generalizar melhor; um λ grande aprende rápido porém arrisca sobreajustar. Por isso, costuma-se fixar um λ pequeno (0.01 a 0.2) e achar o número adequado de árvores via validação. Outros hiperparâmetros de regularização incluem penalizações L1/L2 nos pesos das folhas, `min_child_weight` (tamanho mínimo de peso em folhas no XGBoost) ou `min_data_in_leaf` (LightGBM), que evitam que uma árvore se especialize em poucos exemplos. O tamanho máximo da árvore (profundidade ou número de folhas) também afeta muito: árvores muito rasas podem subajustar, muito profundas podem sobreajustar rapidamente no boosting.

De modo geral, ajustar hiperparâmetros envolve experimentar combinações e avaliar em um conjunto de validação ou via validação cruzada. Ferramentas automatizadas de busca em grade (Grid Search) ou busca aleatória podem ajudar. Por exemplo, podemos realizar uma busca em grade simples para uma árvore de decisão:

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.tree import DecisionTreeClassifier
3
4 param_grid = {
5     'max_depth': [None, 2, 4, 6, 8],
6     'min_samples_leaf': [1, 2, 5, 10]
7 }
8 modelo = DecisionTreeClassifier(random_state=42)
9 grid = GridSearchCV(modelo, param_grid, cv=5)
10 grid.fit(X_train, y_train)
11
12 print("Melhores parâmetros:", grid.best_params_)
13 print("Acurácia (validação cruzada):",
14       f"{grid.best_score_:.3f}")
```

Código-fonte 5 - from sklearn.model_selection import GridSearchCV

Fonte: Elaborado pelo autor (2026)

Suponha que os melhores parâmetros encontrados sejam `max_depth=4` e `min_samples_leaf=2`. Isso indicaria que, no conjunto avaliado, árvores muito

profundas começaram a prejudicar a validação (possivelmente overfitting) e exigir pelo menos 2 exemplos por folha deu uma pequena melhoria de generalização. Em problemas reais, esse processo de tuning frequentemente revela que modelos ensemble (RF, GBM) atingem performances ótimas com configurações intermediárias – nem árvores rasas demais (viés alto) nem liberdade total sem regularização.

Por exemplo, ao tunar uma Random Forest para um problema de churn, poderíamos descobrir que 100 árvores ($n_estimators=100$) já são suficientes (mais que isso não melhora muito), que considerar $\sqrt{d}$ atributos por nó funcionou bem e que forçar folhas com pelo menos 5 exemplos ($min_samples_leaf=5$) aumentou ligeiramente a precisão de teste, por evitar divisões muito específicas. Já para um XGBoost em um problema parecido, poderíamos achar que $learning_rate=0.1$ com $n_estimators=300$ e árvores de profundidade 4 resulta em ótimo desempenho e que profundidades maiores que 6 não trazem ganho, ou até pioram. Cada caso é único e por isso o processo de validação e ajuste de hiperparâmetros é tão importante em machine learning.

Uma ferramenta conceitual útil é observar curvas de treino/validação: por exemplo, aumentar o número de árvores e ver o efeito no erro de treino vs. erro de validação. Métodos de boosting costumam mostrar o erro de treino caindo continuamente com mais árvores, enquanto o erro de validação costuma baixar até certo ponto e depois subir (indicando início de overfitting). A partir dessas curvas ou de métricas como AUC, log-loss, escolhemos o “sweet spot” de iterações. Em Random Forest, por outro lado, o erro de treino geralmente já zera com poucas árvores (porque eventualmente alguma árvore memoriza cada exemplo), mas o erro de validação tipicamente estabiliza ou mesmo aumenta levemente depois de certo número – indicando o ponto onde adicionar árvores não traz mais benefício real.

Em resumo, árvores de decisão e ensembles fornecem um arsenal poderoso ao(a) cientista de dados. Árvores sozinhas são modelos interpretáveis e rápidos, mas podem precisar de poda e cuidado contra overfitting. Ensembles como Random Forest e Gradient Boosting trazem desempenho superior e robustez, ao custo de perder um pouco da interpretabilidade individual (embora possamos extrair importâncias e explicações caso-a-caso com técnicas como SHAP values). Com o domínio desses fundamentos e a prática de tuning, estamos equipados para encarar

problemas complexos de classificação e regressão no mundo real, escolhendo a abordagem adequada para cada situação.



MERCADO, CASES E TENDÊNCIAS

Depois de tanto rigor, vamos conversar sobre alguns casos de uso reais de árvores de decisão e ensembles, além de identificar tendências atuais no mercado. Aqui vale uma pitada de storytelling lúdico, analogias criativas e até algumas figuras para ilustrar ideias.

Caso 1: Redução de Churn em Telecom – o Comitê Salvando Clientes: a empresa fictícia TeleSim sofria com alta taxa de churn (clientes cancelando seus planos de celular). Inicialmente, a equipe de dados criou um modelo de árvore de decisão para prever cancelamentos. A árvore mostrava regras interpretáveis – por exemplo, clientes com mais de duas reclamações e uso de dados baixo eram indicados como alto risco de churn. Mas, quando testada, a árvore obteve acurácia apenas moderada (cerca de 70%) e parecia instável: dependendo do conjunto de treino, a regra raiz mudava (ora era “nº de reclamações”, ora “meses de contrato restantes”).

Para melhorar, a equipe decidiu adotar um ensemble de árvores. Treinaram uma Random Forest com 200 árvores em bootstraps de seus dados históricos e, de cara, a acurácia subiu para 80%. Mais importante, o modelo ficou muito mais estável: qualquer que fosse a amostra de treino, a performance variava pouco e as características mais importantes eram sempre as mesmas (reclamações, tempo de contrato e número de vezes que excedeu a franquia de internet). Ao implementar o modelo, a TeleSim passou a identificar com antecedência 15% mais clientes insatisfeitos, permitindo ao marketing agir (oferecendo promoções ou melhorias) antes que cancelassem. O resultado após seis meses foi 10% menos cancelamentos mensais, o que representou milhões em receita preservada.

Nesse caso, a Random Forest atuou como um comitê de especialistas votando sobre o churn de cada cliente – lembra da analogia do “conselho” de árvores? Pois é. E a interpretabilidade global não foi perdida: a equipe pôde extrair a importância das variáveis e descobriu insights como “clientes que não usam muito a internet mas têm pacotes grandes têm alto risco” – o que motivou a criação de pacotes personalizados. Este exemplo ilustra porque, em telecomunicações e mídia

digital, modelos ensemble são amplamente empregados para entender e prever comportamento de clientes.

Hoje, é comum pipelines em que um XGBoost ou LightGBM roda diariamente para atualizar as probabilidades de churn de milhões de usuários, alimentando dashboards e ações de CRM. Esses modelos se tornaram peça-chave de estratégias de Customer Success por conseguirem aliar boa acurácia preditiva com algum nível de interpretabilidade (via feature importance), balanceando bem decisão automatizada e insight de negócio.

Caso 2: Detecção de Fraudes Financeiras – Floresta vs. Fraudadores: em um grande banco, a equipe de segurança queria aprimorar a detecção de transações fraudulentas em cartões de crédito. Modelos lineares simples davam muitos falsos negativos (fraudes não detectadas) ou falsos positivos (bloquear clientes legítimos) – não conseguiam capturar padrões complexos de fraude. O time então implementou um sistema de duas camadas: primeiro, um Random Forest rápido filtrava transações potencialmente suspeitas; em seguida, um modelo mais pesado de Gradient Boosting refinava a decisão.

O Random Forest foi treinado com milhões de registros de transações históricos, com sinalização de fraude (baseado em chargebacks confirmados). Essa floresta atuava como um screener em tempo real: em milissegundos, ela atribuía um score de risco a cada nova compra. Se o score excedesse certo limiar, a transação era enviada para análise aprofundada. Nessa segunda fase, um modelo de XGBoost – que foi treinado incluindo variáveis mais detalhadas (como sequência de compras, IP de origem, geolocalização) – calculava uma probabilidade de fraude mais acurada. Esse XGBoost era atualizado semanalmente usando os casos confirmados mais recentes, mantendo-se adaptado às novas táticas de fraudadores.

O resultado? A detecção de fraudes melhorou drasticamente. Antes, o banco detectava ~60% das fraudes com uma taxa de 2% de falso positivo. Com o novo sistema, chegou-se a 85% das fraudes detectadas, mantendo falsos positivos em ~3%. A combinação Random Forest + Boosting funcionou bem porque a floresta rapidamente reduz a dimensionalidade do problema (descartando 95% das transações seguras com quase nenhuma fraude perdida) e o booster foca nos casos realmente ambíguos, onde padrões sofisticados fazem diferença.

Além disso, ao inspecionar a importância das variáveis no XGBoost, a equipe descobriu indicadores de fraude antes despercebidos – por exemplo, a sequência de tentativas de compra declinadas em curto intervalo era um sinal forte. Isso levou a ajustes nas regras de negócio também (como bloquear cartões após 5 declínios em 10 minutos). Esse caso de uso reflete uma tendência no setor financeiro: uso de ensembles tanto pela capacidade preditiva em dados tabulares heterogêneos (transações, perfis) quanto pela possibilidade de extrair justificativas para cada alerta (usando técnicas de explicabilidade, hoje alguns bancos usam SHAP para explicar a um analista humano por que o modelo deu aquele alerta de fraude).

Caso 3: Competição e Tendências – O Reino do Boosting em Data Science: no cenário de competições de Data Science (como Kaggle), métodos de ensemble – especialmente Gradient Boosting – se tornaram reis. Um caso emblemático ocorreu na competição Kaggle “Otto Group Product Classification” em 2015: centenas de equipes tentavam classificar produtos de e-commerce em categorias. Ao final, quase todas as soluções de topo eram ensembles de modelos e em particular o uso de XGBoost era onipresente. De fato, a partir de 2015, houve uma tendência tão forte de vitórias do XGBoost que em tom de brincadeira a comunidade dizia: “There is no Kaggle contest that XGBoost cannot win”. Essa dominância levou empresas e ferramentas de AutoML a incorporarem boosting de árvores como componente padrão. Hoje, bibliotecas como scikit-learn incluem implementações otimizadas de Random Forest e Gradient Boosting e frameworks de AutoML (H2O, auto-sklearn etc.) quase sempre terminam suas buscas entregando um modelo ensemble (às vezes até um stacking de vários boosters e florestas).

No mercado, isso se traduz em um conhecimento: para muitos problemas de dados tabulares estruturados (planilhas com diversas colunas numéricas e categóricas), os modelos baseados em árvores continuam sendo estado da arte, superando inclusive redes neurais profundas, que brilharam mais em dados não estruturados (imagens, áudio, texto).

Ou seja, nas empresas de setores como varejo, saúde, telecom ou manufatura, onde os dados principais são tabelas de clientes, transações, medições, a tríade árvore/RandomForest/Boosting é ferramenta fundamental. Uma tendência recente é o surgimento de algoritmos de explainable boosting – por exemplo, a técnica EBM (Explainable Boosting Machine) do Microsoft Research, que treina

modelos aditivos com pequenas árvores e monotonicidades impostas, buscando juntar o melhor dos dois mundos: a interpretabilidade próxima à de um modelo linear com a performance de um ensemble.

Outra tendência importante é a preocupação com interpretabilidade e viés algorítmico. Modelos do tipo Random Forest e Boosting são considerados caixa-preta frente a uma única árvore de decisão ou uma regressão linear. Em setores regulados (financeiro, saúde pública) e em aplicações críticas (ex.: um modelo decidindo se um réu obtém liberdade condicional ou não), há restrições ao uso de modelos não explicáveis.

Isso impulsiona duas frentes: (1) desenvolvimento de métodos para explicar modelos ensemble pós-treino (como os já mencionados valores de Shapley/SHAP, ou LIME, que fornecem explicações locais para predições) – hoje é comum você rodar um Random Forest para decidir um tratamento médico, mas junto entregar quais variáveis pesaram naquela predição específica para o(a) médico(a) entender; e (2) a preferência por modelos interpretáveis simplificados em certos casos – por exemplo, algumas fintechs optam por uma árvore de decisão curta ou lista de regras decisórias para motivos de compliance, mesmo sabendo que perderão um pouco de acurácia em relação a um XGBoost. Pesquisas atuais em fairness também estudam como ensembles podem amplificar ou diluir vieses (por exemplo, se os dados carregam preconceitos, um ensemble tende a replicá-los; porém, por serem mais complexos, podem também mascará-los – dificultando detecção).

Em resumo, as técnicas de árvores de decisão e ensembles tornaram-se onipresentes: estão em sistemas de recomendação de produtos, na priorização de leads de vendas (um XGBoost pode priorizar quais clientes têm maior chance de conversão, orientando equipes comerciais), no diagnóstico assistido por IA (auxiliando médicos a avaliar risco de doenças baseado em exames), no ajuste de políticas públicas (um governo pode usar modelos ensemble para estimar quais regiões têm maior risco de desmatamento ilegal, por exemplo).

A tendência é que elas continuem relevantes, seja como ferramentas independentes ou em conjunto com outros avanços – por exemplo, combinando redes neurais e árvores em cenários de dados mistos (tabular + imagens), ou usando saídas de modelos de linguagem como features em um modelo de árvore. Mesmo com todo o hype em torno do Deep Learning, atualmente ainda é comum

ouvir de profissionais que seu “feijão com arroz” em Machine Learning para dados estruturados envolve treinar algumas Random Forests e Gradient Boosting e comparar. E, frequentemente, isso resolve o problema com excelência.

EMAP

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a fundo o universo de Árvores de Decisão e Métodos Ensemble. Recapitulando os principais pontos, aprendemos como as árvores de decisão tomam decisões dividindo recursivamente os dados com base em critérios de impureza como Índice Gini e Entropia, buscando maximizar o ganho de informação a cada divisão. Vimos exemplos de cálculos de impureza e entendemos que uma árvore cresce até seus nós serem “puros” ou até atingir critérios de parada.

Discutimos o problema do sobreajuste em árvores muito profundas e as técnicas para combatê-lo. Abordamos parâmetros de regularização (profundidade máxima, mínimo de amostras por folha etc.) e o conceito de poda (pruning) para cortar ramos que adicionam complexidade desnecessária. Destacamos que há um balanço viés–variância: árvores mais simples têm menos variância mas mais viés e vice-versa.

Introduzimos o conceito de ensembles como combinação de múltiplos modelos para formar um modelo mais forte. Especificamente detalhamos dois métodos principais: Bagging e Boosting. No bagging, modelos (como árvores) são treinados em paralelo, com amostragem bootstrap, e suas previsões são agregadas para reduzir a variância e melhorar a estabilidade. No boosting, modelos são treinados em sequência, cada um corrigindo erros do anterior, visando reduzir o viés e melhorar gradualmente a performance. Discutimos em profundidade as diferenças e as circunstâncias em que cada um brilha, resumindo comparativamente bagging vs. boosting.

Estudamos a Floresta Aleatória, exemplo de sucesso do bagging aplicado a árvores. Entendemos como ela adiciona aleatoriedade na seleção de atributos para tornar as árvores decorrelacionadas, resultando em um ensemble poderoso que equilibra bem acurácia e robustez. Vimos que o Random Forest é fácil de usar (pouco tuning crítico) e fornece ferramentas como erro out-of-bag e importância de variáveis, sendo muito utilizado na prática.

Aprendemos o conceito de boosting por gradiente, onde árvores sucessivas são ajustadas aos resíduos do modelo atual, melhorando iterativamente o ajuste. Mencionamos algoritmos populares (AdaBoost, XGBoost, LightGBM) e que, embora

mais complexos de ajustar, os métodos boosting frequentemente alcançam as melhores acurácias em problemas tabulares. Também salientamos a necessidade de regularizar e escolher hiperparâmetros adequadamente para evitar que o boosting sobreajuste (via learning rate, número de árvores, profundidade controlada etc.).

Abordamos os principais hiperparâmetros de árvores e ensembles – profundidade, número de árvores, taxa de aprendizado, fração de amostragem etc. – e como realizar técnicas de tuning para determiná-los, usando validação cruzada e busca em grade. Vimos um exemplo de como encontrar a profundidade ótima de uma árvore via GridSearchCV. Reforçamos a importância de avaliar modelos em dados não vistos (validação/teste) para assegurar que a melhora em treino se traduz em generalização.

Durante a aula (e especialmente na seção de cases), discutimos exemplos práticos e tendências. Mencionamos aplicações de árvores e ensembles em diversos domínios: previsão de churn de clientes em telecom, detecção de fraudes financeiras, segmentação de clientes em marketing, diagnóstico médico assistido, entre outros. Observamos que, para dados estruturados, ensembles de árvores continuam sendo uma referência pela combinação de desempenho e certa interpretabilidade. Também comentamos sobre a interação desses modelos com demandas atuais – como interpretabilidade (uso de SHAP, seleção de modelos mais simples em cenários críticos) e integração com novas tecnologias (AutoML, híbridos com deep learning).

Em síntese, você deve sair desta aula compreendendo quando e por que usar árvores de decisão, como melhorá-las com ensembles e quais são as vantagens e cuidados dessas abordagens. Árvores fornecem um modelo de decisão hierárquico intuitivo; ensembles elevam a potência preditiva dessas árvores a níveis superiores, conquistando uma posição de destaque tanto em competições de Machine Learning quanto em soluções de negócios no mundo real. Armado(a) com esse conhecimento, você estará apto(a) a identificar problemas em que uma árvore simples pode ser um bom ponto de partida e situações em que vale a pena convocar uma “floresta” inteira ou uma série de “boosters” para atingir a performance necessária!

REFERÊNCIAS

BREIMAN, L. Random Forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001. Disponível em: <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>. Acesso em: 23 fev. 2026.

GOMES, P. C. T. **Métodos de Ensemble Learning: Bagging, Boosting e Stacking**. 2024. Disponível em: <https://www.datageeks.com.br/metodos-de-ensemble-learning/>. Acesso em: 23 fev. 2026.

GOMES, P. C. T. **O que é Floresta Aleatória e Como Aplicar esse Algoritmo?** 2024. Disponível em: <https://www.datageeks.com.br/floresta-aleatoria/>. Acesso em: 23 fev. 2026.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. **The Elements of Statistical Learning: Data Mining, Inference, and Prediction**. 2. ed. New York: Springer, 2009. Disponível em: <https://hastie.su.domains/ElemStatLearn/>. Acesso em: 23 fev. 2026.

HONÓRIO, R. **O que é Random Forest e como usar em Machine Learning**. 2025. Disponível em: <https://hub.asimov.academy/blog/o-que-e-random-forest/>. Acesso em: 23 fev. 2026.

JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. **An Introduction to Statistical Learning**. New York: Springer, 2013. Disponível em: <https://www.statlearning.com/>. Acesso em: 23 fev. 2026.

SHEIKH, C. **Medindo Entropia e Gini: Compreendendo Árvores de Decisão**. 2025. Disponível em: <https://studyeasy.org/medindo-entropia-e-gini/>. Acesso em: 23 fev. 2026.

WIKIPÉDIA. **Floresta Aleatória**. 2024. Disponível em: https://pt.wikipedia.org/wiki/Floresta_aleat%C3%B3ria. Acesso em: 23 fev. 2026.

PALAVRAS-CHAVE

Árvores de decisão. Floresta Aleatória. Gradient Boosting.

EMAP



POSTECH